THE KOSELLECK MACHINE

Pipeline Manual

What each of the eight pipeline stages does internally, why it is built the way it is, and where it still falls short of what it should do.

REPOSITORY	koselleck-networks
DOCUMENT	docs/pipeline_manual.tex → docs/pipeline_manual.pdf
COMPANION	docs/overview.pdf (setup and run commands), docs/method.pdf (the scientific argument)
COMPILED	September 9, 2026

Contents

1 What this document is

`docs/overview.pdf` tells you what this project is and how to install and run it. `docs/method.pdf` argues the scientific case, with the resolution sweep and the labeling prompt reproduced in full. `README.md` in the repository root carries the same run commands as `docs/overview.pdf` for a reader who only has GitHub open. None of those three walks through what each stage of the pipeline does inside, or why it is built that way and not some simpler alternative. This document is that walkthrough, stage by stage, in run order: parse, repair OCR, bucket into periods, train embeddings, build the network, detect communities, label communities, measure change - plus the region-split variant these eight stages loop over, and the grounded chatbot built on top of their output once they have run.

Two things apply across several stages. Every stage is a standalone script under `src/` that reads its input from the previous stage's output directory. Two of them, training embeddings and building the network, skip a variant whose output file already exists instead of overwriting it: a script has to be told to delete the file before it will redo the work, since retraining a period's embeddings or rebuilding its network is expensive, and an accidental rerun should not throw that away for free. The other five stages always overwrite. Second, most stages loop not only over the twenty period slices but over every **variant**: the combined corpus, plus one independently trained pass per region (American, British) found on disk. §10 explains what a region contains and why the project keeps them apart.

2 Stage 1: Parse

`src/parse_tcp.py` turns raw source files into one flat `processed/all_docs.jsonl`, one JSON record per document: `region`, `source`, `doc_id`, `year`, `text`. Bucketing into period windows is a separate, later stage (`bucket_periods.py`), so a period-boundary change ordinarily only means re-sorting text this stage already extracted, not rerunning it. OCR repair works differently: it runs inside this script, on text read directly from the raw archives, so a changed OCR rule does mean rerunning `parse_tcp.py` (§3).

Two source formats feed the same output file. The first is the Text Creation Partnership's P4 XML: TCP marks up EEBO, ECCO, and Evans by hand, and `extract_text` walks the XML tree collecting text between block-level tags (paragraphs, divisions, line breaks) while treating inline editorial tags, an illegible letter marked `<GAP/>` for instance, as invisible, not as word boundaries. Getting this distinction wrong once produced whole communities of nothing but split-word fragments, which is why the tag list in `BLOCK_TAGS` is spelled out rather than inferred. The publication date comes from the bibliographic

PUBLICATIONSTMT under SOURCEDESC, not the sibling field under FILEDESC, which records when TCP processed the file, not when the book was printed. A document with no extractable year is dropped: it cannot go into a time bucket and is useless downstream regardless of source.

The second format is the British Library's digitised nineteenth-century collection, read as JSONL pages inside per-decade `.tar.gz` archives. `iter_bl_records` groups pages by volume and takes the year from whichever page in the volume carries a date field; it keeps a volume only if its *first* page is tagged English, not a check of every page in it, so a volume that switches language partway through is not caught. A fixed set of archives, `BL_SKIP_ARCHIVES`, is skipped outright: the pre-1800 decades, since TCP's own transcription already covers that span more cleanly, and `unk.tar.gz`, whose records carry no date at all and would be dropped anyway. Neither TCP nor the British Library supplies clean text on its own. TCP's transcribers resolved most OCR-class problems by hand, which is why Stage 2 barely touches TCP text; the Library's scanned pages carry uncorrected OCR damage that Stage 2 exists to address.

A third source of noise sits inside otherwise well-formed documents: non-English text mixed into what is nominally an English corpus, found as an entire community of French words (`mort`, `ilz`, `eux`, `prendre`, `estre`) and a separate one of Welsh (`nefoedd`, `ddim`, `iddo`) surfacing in the trained networks. A first attempt ran `langdetect` per paragraph. Profiling killed it: the check alone cost 180 milliseconds per document, ninety-seven percent of this stage's total running time, roughly five hours across the full corpus, to catch a measured 0.09 percent of paragraphs. The shipped version instead classifies a whole document once, dropping it outright only above 90 percent confidence that it is not English, bringing the cost down to about 27 minutes for the full corpus. Whole-document classification also resolved the French case more thoroughly than expected: what looked like a short quotation embedded in an English document turned out, on inspection, to be an entire Year Book law report, written wholly in Law French as English case law was until 1731, which the coarser per-document check caught cleanly where a per-paragraph check would have had to catch every paragraph separately. A short foreign phrase inside an otherwise English document is caught by neither version; that is treated as the same class of problem as the citation-abbreviation tokens still passing through into the network (§6), word-level noise this stage does not try to solve.

A corpus in neither format can still be added without touching anything downstream of this stage. Already-valid TCP P4 XML needs no code change at all: drop the zip files under `<data_root>/corpus/<a-region-name>/<a-source-name>/` and rerun the pipeline, since `discover_sources` finds new regions and sources by scanning the folder tree, not from a hardcoded list. Any other raw format needs one new reader function beside

`iter_xml_members`, emitting the same `{region, source, doc_id, year, text}` record the TCP path already writes; `bucket_periods.py` through `metrics.py` and the webapp read only that normalised record and need no change. `docs/overview.pdf`'s Bringing your own corpus section gives the full worked version of this contract.

3 Stage 2: OCR Refinement

`src/ocr_refinement.py` repairs OCR-specific damage in a source's text before it is written to `all_docs.jsonl`. It is meant to run for any source listed in `config.yml`'s `ocr_sources` (today, just `[british, bl_19c]`), but the code that decides whether to call it checks the literal tuple `("british", "bl_19c")` against that list, not general membership in it: adding a new OCR'd TCP-format source to `ocr_sources` would not by itself route it through this stage, contrary to what `config.yml`'s own comment implies. TCP is hand-transcribed and never reaches this stage regardless. The module runs three repair steps in a fixed order.

3.1 Layer 1a: Line-break hyphen repair

Rejoins a page's line-break hyphen where it survived into the plain text as a literal - followed by whitespace (`"utrum- que"`). A hyphenated compound never has whitespace between the hyphen and the next letter, so this is a plain regular expression with no wordlist involved, and it cannot mistake a compound for a spacing artifact. A sampled decade archive showed roughly 28% of pages carrying at least one instance.

3.2 Layer 1b: Bare-space split repair

Handles the harder case where the OCR engine dropped the hyphen entirely and left a bare space (`"par ticulars"`), with no marker left to detect it by. It merges two adjacent words only if the merged form is a real word in a reference wordlist, and at least one of the two halves is not independently a real word on its own: that second condition is what keeps a phrase such as `"well known"` untouched, both halves already being real words, at the cost of missing splits where both halves happen to be real words too, such as `"in put"` for `"input"`. The wordlist is built from that period's own already-extracted TCP text, and only from TCP text: `parse_tcp.py` keeps one running word count per period, updated as TCP documents are read, and hands the British Library loop whichever period's count matches a document's date, or an empty one if none does. TCP ends at 1800, so every period entirely after that date gets an empty wordlist, and `repair_bare_space_splits` returns an empty-wordlist call's text unchanged: Layer 1b does nothing across most of where the British Library's OCR damage actually

sits. It was validated by hand in the one period a wordlist existed to test it against, 1790-1810, after temporarily adding a small TCP slice so that bucket would have coverage at all: 142 merges made in the first 120 changed documents, all confirmed correct, following a first validation attempt that turned out to be a false positive, a detection script checking whether two fragments existed anywhere in a document rather than whether they sat adjacent at the claimed merge site. Periods entirely after 1800 remain both unrepaired by Layer 1b and untested.

3.3 Layer 1c: Long-s misreads

This is a named, called function that does nothing. The one OCR'd source in this pipeline postdates the long s's disappearance from English printing, so there is no text anywhere in the corpus to calibrate a character-confusion rule against; a rule guessed without that calibration would risk corrupting every letter *f* in the real text for no benefit. The function stays in place so a future pre-1800 OCR'd source only needs it filled in, not the pipeline rewired around it.

4 Stage 3: Bucket into Periods

`src/bucket_periods.py` sorts every extracted document into a twenty-year time window and writes one plain-text file per period, one document per line, plus a matching `<period>_<region>.txt` for every region present. The windows are uniform across the whole 1510 to 1910 span. Fine resolution only around the Sattelzeit would bias the method toward finding a reorganization only where one is already expected, and Heuser's own earlier chapter found a second, unrelated acceleration window around 1905 to 1925 that a Sattelzeit-only resolution could not have detected. The 1510 start, not 1500, is a deliberate ten-year shift: it puts both 1770 and 1830 exactly on a period boundary instead of in the middle of one, so the transitions that open and close the Sattelzeit are each a literal edge between two periods, not an approximation.

Twenty years is a default, not a swept parameter: `config.yml`'s `periods` table is a plain, user-editable list, so any width, or non-uniform widths, is a configuration change. Twenty is a legibility default, chosen as roughly one literary generation rather than measured against this project's own data; unlike Stage 4's hyperparameters and Stage 5's `top_k` (§5, §6), it was never tuned against this project's own data, and it is exposed as a knob so a different research question, or a different view of how fast a generation turns over, can pick a different width without touching the pipeline. A handful of other settings in this pipeline rest on the same kind of untested first guess - `ocr_refinement.py`'s `min_merged_frequency`, `score_threshold`, and `freq_weight`, and Stage 6's `max_community_size` and `band_width`: none of them has been tested, so there is no

evidence either way.

5 Stage 4: Train Embeddings

`src/embeddings.py` trains one Word2Vec model per period, and per region-split variant, on that variant's own bucketed text: a two-hundred-dimension vector space in which words that tend to appear near each other end up positioned near each other. Current settings are `window=5`, `min_count=50`, `vector_size=200`, `epochs=10`. A variant with fewer than twenty documents is skipped outright; a handful of idiosyncratic books is not a period-level signal.

Its tokenizer also excludes roman numerals (`xxiiij`, `xxvij`) before a word ever reaches the vocabulary this stage trains on, a fix shipped 2026-09-01 after these numerals turned up clustering tightly enough by co-occurrence to form their own communities downstream. The check validates a token positionally against how a real roman numeral is built, rather than pattern-matching letters, so it does not also catch real English words built entirely from roman-numeral letters (`civil`, `mill`, `vivid`). Latin citation-apparatus abbreviations are a related but distinct case, untouched by this fix; §6 covers what they are and why they still reach the network.

These two headline parameters, `min_count` and `window`, went unvalidated until a 2026-08-28 robustness sweep tested four alternative values of `min_count` and three of `window` against eight pilot periods bracketing the Sattelzeit, screening at three of the fifteen mandatory resolutions instead of the full sweep. No alternative produced a cleaner Sattelzeit-localized signal than the defaults, so the defaults were kept; that is the entire content of the decision, and it is easy to misread as a confirmation of the underlying claim. It is not one. The sweep surfaced an unresolved picture: the reorganization signal under every tested configuration rises gradually from early to late periods instead of spiking and returning inside the Sattelzeit window, and shared vocabulary between consecutive periods grows roughly ninefold across the run, tracking the British Library's bulk entry into the corpus in the same late periods that still carry unrepaired OCR noise. Part of the late-period rise in the project's headline metric could be that noise, not semantic reorganization; the sweep neither confirms nor rules that out, and the full fifteen-resolution version of it, which CLAUDE.md's hard constraint requires before any reorganization claim can rest on it, has not been run.

6 Stage 5: Build the Network

`src/network.py` turns one period's embedding space into a graph: every remaining word becomes a node, and an edge connects each word to its `top_k` nearest neighbours by

cosine similarity. A fixed similarity threshold was tried first and dropped: at 0.3, the resulting graphs were dense enough to be unusable, roughly ten percent of all possible edges present, tens to hundreds of millions of edges per period at this vocabulary scale, too large for community detection and too large to store. `top_k=15` bounds the graph to roughly fifteen edges per word regardless of vocabulary size, and this value was itself tested on 2026-08-31 against the same eight pilot periods Stage 4's sweep used, at values of 5, 10, 20, and 30, screening at the same three resolutions rather than the full fifteen, the same limitation as Stage 4's sweep and not yet a paper-ready robustness claim on its own. Three of the four alternative values produced the same qualitative picture as the default, a gradual rise rather than a Sattelzeit-localized spike; the fourth, `top_k=5`, sat uniformly higher across every transition, early periods included, which reads as a sparser graph being generally less stable under Leiden, not as specifically more sensitive to the Sattelzeit.

Before edges are built, two kinds of word are excluded from becoming nodes at all: function words and early-modern grammatical variants (`hath`, `doth`, `thou`, and similar) in a fixed `STOPWORDS` set, and any single-letter token, which in this corpus is always either a stopword already, a Latin citation abbreviation, an interjection, or an OCR artifact. Neither carries the conceptual content this project measures. This exclusion is still incomplete, though narrower than it was: roman numerals no longer reach this stage at all, excluded upstream during Stage 4's tokenization (§5), but Latin citation-apparatus abbreviations that are not themselves roman numerals (`capvt`, `deute`, `regu`) still pass through into the network from early-modern, hand-transcribed periods, correctly transcribed originals, not OCR damage, and cluster tightly enough by co-occurrence to form their own communities. The labeling stage (§8) already names these accurately, as *Structural / Uncertain* clusters such as *Biblical Book & Chapter Citations*, but the fix belongs here, at the stage that decides what counts as a node, and is not yet built.

Two further capabilities exist in this stage's code and ship off. `use_gpu` offloads the cosine-similarity computation to a GPU via `cupy`; a benchmark against the largest model this pipeline has built, 171,382 words, measured a modest 1.18× speedup, confirming this stage is not today's bottleneck. The capability is banked for a larger future corpus. `pos_filter` restricts which words can become nodes to those tagged noun or adjective, Jamie's proposal for a different problem: without it, several of a period's largest communities are pure grammatical clusters, verb-conjugation groups with no topical content. The one direct before-and-after comparison run so far, on one period, 1750-1770, Combined region, at one resolution, 4.0, confirmed the fix works there, turning grammatical clusters into topical ones, including a vice-words and a virtue-words community that read as near mirror images of each other, but it is not a clean win: removing verb forms also let a new community of pure person-names emerge, a named-entity cluster rather than a grammatical or a topical one. The filter

needs a per-period word-to-category lookup table before it can run for a given period; where that table is missing, `network.py` prints a warning naming the period and builds that period unfiltered instead of failing. The table exists for TCP periods through a part-of-speech-tagged release of the same corpus, roughly 95% coverage of this project's vocabulary, lower at 87% for 1790-1810, but the pre-1700 portion of that release has not been downloaded, and the British Library tables are still paused mid-build.

7 Stage 6: Detect Communities

`src/community.py` runs the Leiden algorithm over each period's network to find communities: groups of words more tightly connected to each other than to the rest of the graph. There is no fixed number of communities anywhere in this pipeline; it is always an output of the algorithm.

Leiden takes a **resolution** parameter trading community size against community count, and every quantitative reorganization claim this project makes has to hold at all fifteen fixed resolution values, 0.1 through 16.0, not just whichever looks cleanest. This is CLAUDE.md's one hard rule: picking a single favourable resolution after seeing the result would let the analyst choose the answer instead of measuring it.

That fifteen-value sweep answers the scientific question but produces partitions useless to show a reader directly, since at low resolution a period's largest community can hold thousands of unrelated words. A second, separate resolution decides what the webapp displays and what the labeling stage reads, picked independently for each variant rather than once globally. On 2026-08-27 an automatic ascending scan replaced the earlier single hand-picked constant, 4.0, that had forced every early, easy period up to whatever the hardest late period needed; the following day, that global scan was itself replaced by the current per-variant band-bisection, since forcing every variant to one shared value carried the same problem the hand-picked constant had, only automated. It works by starting from a seed resolution and doubling or halving until it brackets the lowest resolution at which no community exceeds a fixed 1,500-word cap, then bisecting within that bracket until a passing value lands within 300 words of the cap or the bracket narrows past a safety floor. There is no hardcoded ceiling; the doubling search finds its own upper bound, so a larger future corpus needing resolution 40 instead of 16 does not require anyone to notice and raise a constant by hand.

This mechanism shipped with a bug, caught only once it ran against the full rebuilt corpus rather than the smaller test set it was checked against in 2026-08-28. `1650-1670_american`, 840 words total, never exceeds the 1,500-word cap even at the lowest resolution tested, so the bisection routine's search for a resolution that fails the

cap never found one, and the code crashed expecting a failing lower bound that was never going to exist. The fix: when no resolution ever fails the cap, the lowest resolution actually tested is used directly, with no bisection needed.

This mechanism, bug fix included, has since run against the full rebuilt corpus, 2026-09-02, alongside the roman-numeral exclusion and the foreign-language filter above, producing the community structure every later stage in this document now describes.

8 Stage 7: Label Communities

`src/extract_community_words.py` and `src/label_communities.py` turn each period's bare Leiden communities into plain-English labels a historian can read, through a CSV-based workflow meant to be run, and corrected, by a person rather than trusted blindly from a single model call. The system prompt these scripts send lives in the repository as `src/prompts/community_labeling_v3.md`, parsed at runtime rather than duplicated as a string constant in code; the fit-check prompt referenced below lives at `src/prompts/label_fit_check_v2.md`.

8.1 Label inheritance and the *system/stayed* contradiction

A community's label is not generated independently every time its period comes up. `generate` walks periods in chronological order and, for each community, first checks whether the same Hungarian alignment `metrics.py` uses to compute **migration_fraction** (§9) maps it back to a specific predecessor community that already has a label; that check is free, since the alignment itself needs no model call. Whether the inherited label is then actually reused is not free. An earlier version generated each period's label independently from just that period's own top words, and produced a publicly visible contradiction: the word *system* was flagged as having *stayed* in the same community between two consecutive periods, correctly, by the same alignment `metrics.py` already trusted, while the community's displayed label changed anyway, from *Books, Manuscripts & Learned Vice* to *Classical Authors & Scholarship*, because the model reading a slightly different word list described it differently. To a reader that looks like the tool contradicting itself. Inheriting the label whenever the alignment says the community is the same makes *the label changed* and *the word moved* the same event by construction, not two independent judgments that can disagree.

8.2 The two-reader fit check

Verbatim inheritance on its own then created the opposite failure: with no mechanism to force a fresh read, a label could ride unchanged for a community's entire lifetime

while its content drifted completely away from what the label describes. One documented case: a community genuinely holding Dutch and German text at 1510 to 1530 was still labeled *Dutch and German Text* fifteen periods, roughly three hundred years, later, by which point its content had passed through Scots legal prose, classical mythology, and early ecclesiastical history before landing on English church governance, none of it Dutch or German. The current mechanism, and the actual cost this stage pays, is two model reads every time a label would otherwise inherit: both given the same inherited label and the same current word sample, both asked the identical plain yes-or-no question, does this label still fit, independently of each other, so agreement is a genuine redundancy check. Only unanimous agreement skips a fresh read; either reader saying no, or the two disagreeing, forces one immediately.

8.3 Stratified word sampling

The word sample a community shows, to both the fresh-labeling prompt and the fit-check prompt, changed on 2026-08-31. It used to be a flat list of the twenty-five words with the highest network degree in that community, its most central members. That sample only ever shows words tightly bound to the community's core, which hides the opposite case: a community with a coherent-looking core can still have a drifting low-degree tail belonging to a different topic, and a reader who never sees that tail has no way to notice. The word list shown is now a stratified sample by degree rank, not a flat top slice: up to fifteen words from the top of the ranked list, fifteen from around its midpoint, and fifteen from its bottom, forty-five in total, drawn from a membership list that can hold up to 1,500 words, not an exhaustive split of it. The tier size was raised from nine to fifteen per tier on 2026-09-02, kept symmetric across all three on purpose: the failure this sampling change fixed was under-weighing the mid-rank and peripheral tiers in the decision itself, not insufficient signal from the top tier, so giving the top tier more text share would risk reintroducing the same bias by sheer volume. A community small enough that this would show nothing new, forty-five words or fewer, is shown whole instead.

The redo this change required, run across all three regions, caught drift the flat list had been hiding. One community's label, *Latin Liturgical & Hymn Vocabulary*, no longer matched its own core, which had drifted to English virtue-adjectives. A different community kept an OCR-fragment label naming the wrong stripped prefix after its dominant fragment pattern had itself shifted. A third, *Church Fathers & Ecclesiastical History*, had drifted into ordinary English clergy surnames. Those three named cases are the direct evidence that the tiered sample catches drift the flat list missed. The aggregate fit-check failure rate is weaker, mixed evidence rather than confirming: roughly eight percent for Combined against six before the change (on 1,310 checks, not

a small-sample move), but British fell slightly (sixteen against seventeen) and American fell more (forty against fifty, though on only 42 checks, too small to weigh heavily). Only one of the three regions' aggregate rate moved in the expected direction; the case for the change rests on the three named communities, not the aggregate numbers.

8.4 Lane taxonomy and reproducibility

Every fresh label names exactly one lane from a fixed list of fifteen, decided 2026-08-03 from 707 labels already produced across earlier runs, instead of inventing a category per community. The fifteenth lane, *Structural / Uncertain*, covers a community grouped by grammar rather than topic, by a shared non-English language, by proper names, or, in the British Library's nineteenth-century periods, by shared OCR damage: a cluster of prefix- or suffix-stripped fragments rather than words at all.

This project's stated reproducibility standard for these labels is what its internal documentation calls Tier 1 or 2: the same input produces a comparable label, not the same label token-for-token, since a model call is not bitwise-deterministic even against an unchanged prompt. It does not claim Tier 3, exact reproducibility.

8.5 Forcing reasoning before the label

The prompt each fresh read is given changed again on 2026-09-02, independently of the word-sample change above. Reviewing labels at scale surfaced a milder but related failure: a fresh read defaulting to *Structural / Uncertain* without clear evidence the stratified sample's lower two tiers had actually been weighed, reaching for the easiest reading from the top tier alone. The tool schema `call_llm` sends now requires a `reasoning` field ordered before `label` and `lane`, so the model's own generation works through the core, mid-rank, and peripheral tiers, naming a specific word from each, before it commits to a label. A blind test on two communities, run twice each through a fresh no-context agent with the old and new wording, found the old prompt defaulting to *Structural / Uncertain* both times despite its own draft label already naming a real theme, and the new prompt reaching a concrete, evidence-grounded lane both times instead.

8.6 The 2026-09-08 full relabel

A full relabel followed, all three regions, against the fresh community structure the 2026-09-08 rebuild produced (the display-resolution cap lowered to 1,500 words, §7 again): 100 communities in American, 47 read fresh and 53 inherited, 32 in *Structural / Uncertain*; 3,099 in British, 1,319 fresh and 1,780 inherited, 1,227 in *Structural / Uncertain*; 2,981 in Combined, 1,091 fresh and 1,890 inherited, 1,256 in *Structural / Uncertain*. Two-

fifths to a third of every region landing in the fifteenth lane is not a new problem this relabel introduced; it sits inside the range this stage's own documentation already expected, and it does not weaken the reorganization measurement in §9, since that measurement runs on the community partition directly and never reads label text, and a stable, grammar-bound community is more likely to dilute a detected reorganization signal than inflate one. A separate safety fix, `record-fit-checks`, now merges each batch of fit-check answers into the saved file rather than replacing it outright, after an unrelated test script's cleanup step deleted 1,310 prior answers by reusing one path variable for both a scratch file and the production one.

9 Stage 8: Measure Change

`src/metrics.py` is where the pipeline's central question gets a number. For every pair of consecutive populated periods, at every one of the fifteen mandatory resolutions, it restricts to the vocabulary shared by both periods, since each period's embedding model is trained only on that period's own text, and computes three things: normalized mutual information and adjusted Rand index, two label-permutation-invariant measures of how similar the two periods' community structure is, and **migration_fraction**, the fraction of shared words that end up in a different community even after the best possible one-to-one relabeling between the two periods, found by the Hungarian algorithm on the word-overlap contingency table between every pair of communities. This is the same alignment mechanism Stage 7 uses to decide whether a label inherits, applied here to a different question: *how much of the vocabulary structure reorganized*. A pair of periods sharing fewer than twenty words is skipped; a metric computed over that few words would not mean anything.

Run against the full rebuilt and relabeled corpus, 2026-09-04, this produces the same qualitative picture §5's sweep already found at three of the fifteen resolutions, now visible across all fifteen: `migration_fraction` rises gradually across the whole 1510-1910 span rather than spiking inside the Sattelzeit window. In the Combined variant, the 1790-1810-to-1810-1830 transition averages 0.5163 across resolutions, a real rise from the prior transition's 0.4477, but only the second-highest transition in the entire series; the highest, 0.5549, falls earlier, at 1690-1710 to 1710-1730, and coincides with a real drop in shared vocabulary between those two periods, to 10,834 words from 43,021 the transition before, a pattern not repeated anywhere else outside the small-vocabulary noise expected at the very first transition in the series. That coincidence has not been investigated further; a sharp shared-vocabulary shock between two periods is the same class of confound §5 already names for the British Library's late-period entry into the corpus (which cannot itself explain this particular, much earlier transition), now visible directly in this stage's own numbers rather than only inferred from vocabulary

growth.

10 Region split: Combined, American, and British

Most stages loop over three variants per period, not one: **Combined**, trained on every document regardless of source, and one independently trained pass per region the raw corpus contains. **American** is the Evans collection alone, thin before roughly 1630 and again after 1810, matching Evans's own coverage. **British** pools four archives that never get their own separate variant, EEBO phase 1, EEBO phase 2, ECCO, and the British Library's nineteenth-century supplement, all tagged `british` regardless of which of the four a given document came from.

None of the three variants is a blend of the other two. Combined is not an average of American and British; it is its own independent Word2Vec model, trained on the pooled text directly, the same way American and British are each trained on their own restricted text. The region split answers a provenance question, not a dialect one: does mixing the British Library's OCR'd nineteenth-century text next to TCP's clean, hand-transcribed earlier periods change the reorganization signal in a way that traces back to where the text came from, not to anything the Sattelzeit hypothesis predicts. It is not a claim that American and British English are dialects worth comparing on their own terms; the American variant, in particular, is too thin across most of the corpus's span to support that kind of comparison even had it been intended as one.

11 Layer 2/3: The Grounded Chatbot (Ask)

Everything above produces the measurement; this layer answers a question about it in plain language, without retraining anything or re-deriving a number the pipeline already computed. It lives in `src/rag/`, reachable from the webapp's `/chat` route (**Ask**) or directly from the command line via `src/rag/engine.py`. Built and merged 2026-09-08, on top of the fully rebuilt corpus and relabeled communities described above.

11.1 Evidence and reliability tiers

`src/rag/evidence.py` defines the one honesty boundary the whole layer is built around: every fact retrieved for the model to cite is wrapped in an `Evidence` record before synthesis ever sees it, tagged one of three tiers. **MEASURED** is a computed quantity or a community assignment - anything out of `metrics.py` or the Leiden partition itself, the project's actual evidence, never graded or overridden by an LLM. **INFERRED** is an embedding-neighbour reading - cosine-kNN similarity edges, directional and sugges-

tive, not causal, and not co-occurrence. **UNRELIABLE** is a fact drawn from OCR-diluted material (§3) or a community the labeler routed to *Structural / Uncertain* (§8); it must be shown with its caveat, never laundered into a clean claim. The downgrade rule is data-derived, not a hand-maintained list: any period whose window starts at or after 1800 rests on the British Library's OCR text (`OCR_CORPUS_START_YEAR`), and any community in the *Structural / Uncertain* lane is unreliable regardless of how clean the attached number looks. Both checks run in exactly one place, `apply_reliability`, called by the tools layer right after constructing each fact, so the honesty boundary is applied once rather than re-remembered per query. One tier does not map onto its own definition as cleanly as the other two: `label_lookup` (below) is tagged INFERRED not because it is a nearest-neighbour computation, but because a label is one LLM's read of a community's top words, not a checked ground truth (§8) - the weakest thing this tier covers, kept here rather than promoted to its own fourth tier.

11.2 The grounded tools

`src/rag/tools.py`'s `Store` class exposes six tools, each backed by a real DuckDB query rather than free generation: `reorganization_metrics` (MEASURED - NMI, ARI, and `migration_fraction` across the full resolution sweep, §9), `word_neighbors` (INFERRED - cosine-kNN neighbours in one period), `community_trajectory` (MEASURED - a word's community in every period it appears, not just two named ones), `words_that_moved` (MEASURED - the concrete words behind one transition's `migration_fraction`), `compare_neighbors` (INFERRED - neighbour churn between two periods), and `label_lookup` (INFERRED - one community's reading-aid label and lane, §8). Every tool's own schema description states its tier directly, so the model is told, not left to infer, how much weight a fact can carry.

Only `reorganization_metrics` reads the fifteen-value sweep (§7) - the figure a reorganization claim is actually tested against. The other five all resolve a period's community structure at that variant's own auto-picked **display** resolution instead (§7 again), the same partition its labels are computed from: `Store._resolution` looks this up per (region, period) from `label_resolution.json`'s per-variant map, returning `None` - treated as no data, never a silent fallback to an unrelated variant's number - for a pair that never satisfied the size cap. Region defaults to `combined` on every tool but is an optional argument throughout, exactly as its own schema declares.

11.3 The engine loop

`src/rag/engine.py`'s `Engine.run()` is a decompose-retrieve-synthesize loop, capped at eight turns (`MAX_TURNS`): the model reads the question, calls one or more tools, gets back Evidence dicts - never raw tables - and either calls again or writes a final answer.

The loop itself is model-provider agnostic: `OllamaProvider` and `AnthropicProvider` both turn a neutral transcript into the same `{text, tool_calls, stop}` shape, so swapping providers touches no other file. A new tool needs two things added together, both in `engine.py`: a `TOOL_SCHEMAS` entry (name, description, and an `input_schema` naming its parameters) and a same-named method on `tools.py`'s `Store` class - `dispatch()` binds the two purely by name, so the schema and the method are the only two places that need to agree.

`dispatch()`, the pure function mapping one tool call onto a `Store` method, is the offline-testable half of the engine - no model needed to unit-test it. A real bug found testing `qwen2.5:14b` against `llama3.1:8b`: the model called `community_trajectory` with `period_from`/`period_to`, parameters that belong to `reorganization_metrics` instead, got back Python's own "unexpected keyword argument" message, and rather than retrying with the right shape it abandoned `community_trajectory` entirely for the weaker, INFERRED-tier `word_neighbors` and built the whole answer on that. Fixed by validating a call's arguments against its own schema before it reaches the method at all, and - for an argument that belongs to a different tool - naming which one, turning a dead end a model could not parse into a fixable one. A related fix the same day: a numeric argument arriving as the string `"10"` instead of the integer `10` (seen for real from `llama3.1:8b`) is now coerced before it can raise an opaque, unattributable `TypeError` deep inside a slice or `LIMIT`.

11.4 The DuckDB store and a real region-mislabeling bug

`src/rag/build_store.py` builds the store the engine queries from the pipeline's own output - one ingestion function per table (provenance, membership, edges, transitions, labels), each reusable to refresh the store after a period or region is added, without a full rebuild. `ingest_transitions` shipped with a real bug found 2026-09-08: `metrics.py` (§9) writes exactly one `transitions.csv`, covering every region together, with the region tag embedded in `period_from`/`period_to`'s own `_british`/`_american` suffix - but the ingestion function assumed the opposite layout, looking for separate `transitions_british.csv`/`transitions_american.csv` files that never existed. The result: every row it did read was tagged `region='combined'` regardless of its real suffix, so a `region='combined'` query got flooded with irrelevant British/American rows on top of the real combined ones, while `region='british'`/`'american'` queries found nothing at all. Fixed by reading the one real file and re-deriving each region's rows via the same period-pair mapping `metrics.py` used to write them in the first place - the identical pattern `webapp/app.py`'s `get_transitions()` already used.

11.5 Multi-transition questions

`reorganization_metrics` takes an optional `window_from`/`window_to` pair, distinct from an exact `period_from`/`period_to` transition, specifically for a vague question like "did it peak around 1770-1830." One call resolves the whole comparison: every transition inside or closing the named window, plus baseline transitions from well outside it, each already reduced to a summary row (median and range) ahead of its full detail. This exists because of a real, observed failure mode: a model asked to compare several transitions, given one tool call per transition, kept building its answer from only the most recently retrieved evidence, silently dropping the earlier calls' results rather than synthesising across all of them. A single call that already contains the full comparison removes the chance for that failure to happen at all, rather than asking the model to remember to look back.

11.6 Model choice

The chosen model is `qwen2.5:14b` (via Ollama, no API credits), not the smaller `llama3.1:8b` first tried. Live-tested against the same real questions, `llama3.1:8b` wrongly refused an answerable question outright, wrote a fake tool-call JSON block into its own visible answer instead of retrying after a real error (the system prompt explicitly forbids this), and separately answered by describing the tool result's JSON schema back to the user instead of actually answering. `qwen2.5:14b` made valid, honest tool calls throughout the same test set. `AnthropicProvider` is built and available as an optional stronger provider (`ANTHROPIC_API_KEY` required) but has not yet been tested against this corpus.

11.7 The grounding eval

`src/rag/eval/` runs a small, deliberately pointed set of real test questions (`cases.py`, six of them: the headline sweep question, a word's trajectory, the words behind one transition, an OCR-period caveat, a nonsense word, and a question the data cannot answer at all) against the live engine, not a benchmark - each case targets one specific behaviour the tool must get right, including questions the built data genuinely cannot answer, where the correct behaviour is a stated refusal rather than an invented finding. `checks.py` grades each result without a model and without the corpus - purely by comparing the answer text against the Evidence it was given: every number the answer states must trace to a number actually present in the Evidence (`numbers_grounded`), a question with no data must be refused, not answered confidently (`refusal_honoured`), and any UNRELIABLE evidence used must be surfaced with its caveat, not laundered into a clean claim (`caveats_surfaced`). An optional `–judge` flag adds an LLM-as-judge pass on top of these deterministic checks, run via `src/rag/eval/run.py`. The harness itself is

built and was used during the live testing that surfaced the bugs above; no full `run.py` pass-rate has been recorded anywhere in the repository as of this writing, unlike the specific counts reported for every other stage in this document.